LJIET — SEMESTER VII
CYBERSECURITY PRACTICAL SOLUTIONS

Practical record guide for all 23 experiments

Note: Commands and outputs below are examples for an authorized, isolated lab. Replace example IPs, domains, usernames, and paths with instructor-approved values. Never scan, attack, collect credentials from, or test systems without explicit permission. Sample outputs are illustrative and may differ on your machine.

1. Network Mapper (Nmap)

Aim: Perform host discovery, port scanning, service/version detection, and OS detection on an authorized lab.

Requirements: Nmap and an instructor-approved lab network/target.

Procedure:

- Discover active hosts on the approved subnet.
- Scan the assigned target for open ports.
- Detect service versions and attempt OS detection.
- Save results and note defensive recommendations.

Commands / filters:

```
nmap -sn 192.168.1.0/24
nmap 192.168.1.10
nmap -sV 192.168.1.10
sudo nmap -O 192.168.1.10
nmap -sV -O 192.168.1.10 -oN nmap_report.txt
```

Observations / explanation:

- Illustrative output may show 22/tcp open ssh, 80/tcp open http, or other services; actual results depend on the target.
- Close unused ports, patch services, apply firewall rules, and monitor scanning activity.

Result: Host discovery and port/service scanning were demonstrated; findings were saved to a report.

Precautions: Scan only targets and address ranges explicitly authorized by the instructor.

2. Wireshark — Packet Capture and Analysis

Aim: Capture and interpret network packets for troubleshooting and security analysis.

Requirements: Wireshark and designated training traffic.

Procedure:

- Select the authorized interface and start a capture.
- Generate permitted lab traffic, such as pinging the lab gateway.
- Apply filters to inspect ICMP, DNS, TCP handshake, and HTTPS metadata.
- Stop the capture and record observations.

Commands / filters:

```
icmp
dns
tcp.flags.syn == 1 && tcp.flags.ack == 0
tcp.flags.syn == 1 && tcp.flags.ack == 1
tcp.port == 443
```

Observations / explanation:

- TCP three-way handshake: client sends SYN, server responds SYN-ACK, and client sends ACK.
- DNS packets show domain queries and responses. HTTPS captures commonly reveal IPs, ports, sizes, and timing, while application content remains encrypted.
- Capture filters operate during capture; display filters select packets already captured.

Result: Training packets were captured and interpreted.

Precautions: Capture only designated training traffic and do not inspect other users' private communications.

3. John the Ripper — Password Hash Analysis

Aim: Identify supplied sample hash formats and discuss dictionary-attack effectiveness and defenses.

Requirements: John the Ripper and instructor-provided sample hashes.

Procedure:

- Copy only the supplied lab hashes into a file such as sample_hashes.txt.
- Use format-identification utilities or instructor notes to determine likely formats; hash identification is not always definitive.
- Run a bounded test against the supplied hashes only.
- Record recovered test passwords, if any, without including real credentials in the report.

Commands / filters:

```
john --list=formats
john --format=FORMAT sample_hashes.txt
john --show sample_hashes.txt
```

Observations / explanation:

- Replace FORMAT with the confirmed lab hash format supported by the installed version.
- Dictionary attacks are more effective against common, short, reused, or predictable passwords.
- Use unique long passwords, MFA, rate limiting for online login attempts, and salted password hashing such as Argon2id, scrypt, or bcrypt with appropriate parameters.

Result: The supplied sample hashes were analyzed and the effectiveness and limits of dictionary attacks were documented.

Precautions: Do not test hashes obtained from real users or systems without explicit authorization.

4. Whois — Domain Lookup

Aim: Retrieve public domain registration and DNS-related information for an approved domain.

Requirements: whois client or an approved registrar/RDAP lookup service.

Procedure:

- Choose a domain approved by the instructor.
- Query its public registration information.
- Record registrar, registration/expiry dates if available, nameservers, and privacy protection.
- Discuss information-disclosure risks.

Commands / filters:

```
whois example.com
```

Observations / explanation:

- Fields vary by registry and privacy settings; some details may be redacted or unavailable.
- Public registration details can assist asset mapping and targeted social engineering. Use privacy protection where appropriate and avoid publishing unnecessary personal details.

Result: Publicly available registration information was reviewed.

Precautions: Respect applicable laws, registry terms, and the approved scope.

5. Dig — DNS Query

Aim: Retrieve and interpret A, MX, and NS records for an approved domain.

Requirements: Linux/macOS dig utility or equivalent DNS tools.

Procedure:

- Query A, MX, and NS records for the approved domain.
- Record returned values, TTLs, and responding DNS server.
- Compare direct queries with passive sources described by the instructor.

Commands / filters:

```
dig example.com A
dig example.com MX
dig example.com NS
```

```
dig example.com AAAA
dig example.com ANY
```

Observations / explanation:

- A maps a name to IPv4; AAAA maps to IPv6; MX lists mail exchangers; NS lists authoritative name servers.
- Active gathering sends queries to DNS infrastructure. Passive gathering uses already-collected datasets or public records without directly probing the target during the collection. ANY responses are often limited and may not return every record.

Result: DNS records were queried and their purpose documented.

Precautions: Query only approved domains and follow lab rules.

6. theHarvester — OSINT

Aim: Collect publicly available emails, hosts, and subdomains from approved public sources.

Requirements: theHarvester installed and an instructor-approved domain/source scope.

Procedure:

- Select the approved domain and permitted data sources.
- Run a limited collection query.
- Save the results and remove duplicates.
- Assess whether exposed addresses or subdomains increase risk.

Commands / filters:

```
theHarvester -d example.com -b duckduckgo
theHarvester -d example.com -b crtsh
theHarvester -h
```

Observations / explanation:

- Exact source names and flags vary by version; consult the installed help output.
- Publicly exposed staff emails can support phishing, while forgotten subdomains may expose unmaintained services. Verify findings with the asset owner and report responsibly.

Result: A scoped OSINT collection was completed and its risks were documented.

Precautions: Use approved sources and targets only; do not use collected contact details to spam or phish people.

7. Zphisher — Phishing Awareness

Aim: Explain how a fake login page can imitate a legitimate site in an isolated awareness exercise.

Requirements: Instructor-controlled isolated lab, dummy accounts, and explicit permission.

Procedure:

- Explain the concept of a lookalike login page without deploying it against a real service.
- In the instructor-provided sandbox, use a fictional brand and dummy credentials only.
- Demonstrate the user-visible warning signs and how a captured dummy submission would be handled.
- Document impact and prevention.

Observations / explanation:

- Phishing pages exploit trust, lookalike URLs, urgency, and password reuse.
- Defenses include phishing-resistant MFA/passkeys, password managers, user reporting, domain protection, email filtering, and avoiding links from unexpected messages.

Result: The phishing-awareness technique, impact, and defenses were explained in a controlled exercise.

Precautions: Do not expose a page to the public internet, impersonate a real service, or collect real credentials. Follow the instructor's sandbox procedure.

8. Burp Suite Intruder — Attack Types

Aim: Understand Sniper, Clusterbomb, Battering Ram, and Pitchfork using an authorized training target and test data.

Requirements: Burp Suite and a deliberately vulnerable lab target.

Procedure:

- Configure the browser to use the Burp proxy and install the lab certificate only if needed for the local lab.

- Intercept a permitted test request and send it to Intruder.
- Mark test positions and use a small dummy wordlist.
- Compare the payload placement strategies and document defensive controls.

Commands / filters:

```
Sniper: one payload position is tested at a time.
Battering Ram: the same payload is inserted into all marked positions.
Pitchfork: multiple payload sets are paired by index.
Clusterbomb: tries combinations across payload sets; combinations can grow rapidly.
```

Observations / explanation:

- Use Sniper for one parameter at a time; Pitchfork for paired values; Battering Ram for the same value in multiple fields; Clusterbomb for combinations.
- Defenses include rate limiting, account lockout or throttling where appropriate, server-side validation, strong authentication, monitoring, and secure session handling.

Result: Intruder attack-type behavior was compared with dummy data.

Precautions: Only send requests to the designated lab; keep request counts low and avoid real accounts.

9. Sublist3r — Subdomain Enumeration

Aim: Enumerate subdomains of an authorized target and save the results.

Requirements: Sublist3r installed and an approved domain.

Procedure:

- Run enumeration for the approved domain.
- Save results to a text file.
- Review results with the asset owner and identify stale or unexpected hosts.
- Document remediation recommendations.

Commands / filters:

```
sublist3r -d example.com
sublist3r -d example.com -o subdomains.txt
sublist3r -h
```

Observations / explanation:

- Flags and source availability may differ by version.
- Inventory all discovered hosts, remove abandoned DNS entries, secure exposed services, and monitor certificate-transparency and DNS changes.

Result: Subdomains were enumerated and the output file was recorded.

Precautions: Do not probe or scan discovered hosts unless that activity is separately authorized.

10. DirBuster — Directory/File Discovery

Aim: Discover directories and files on an authorized training website.

Requirements: DirBuster and an intentionally vulnerable training web server.

Procedure:

- Enter the lab URL and choose an instructor-approved wordlist.
- Set a conservative thread count and run the scan only against the lab.
- Record interesting status codes and paths, then validate them manually.
- Identify the VAPT phase and recommend fixes.

Commands / filters:

```
Example wordlist: the wordlist supplied by the lab/instructor.
```

Record: target URL, wordlist filename, scan settings, discovered paths, status codes, and timestamp.

Observations / explanation:

- This is web content discovery, generally part of reconnaissance or the discovery phase of a VAPT assessment.

- Avoid exposing backups, configuration files, source maps, and admin interfaces; disable directory listing and enforce access controls.

Result: Directory/file discovery was completed against the training site and findings were documented.

Precautions: Never run directory discovery against a public site without written authorization.

11. Shodan — Internet-Exposed Asset Search

Aim: Search for exposed devices, ports, and publicly reported vulnerabilities and recommend defensive actions.

Requirements: Shodan account or public interface; approved organization/domain/IP scope.

Procedure:

- Search only assets you own or are authorized to assess.
- Record query, date, host, visible ports/services, and any vulnerability indicators.
- Validate results with the asset owner because records may be stale or false positives.
- Prioritize remediation by exposure and severity.

Commands / filters:

Example query pattern: hostname:example.com

Example query pattern: net:203.0.113.0/24

Observations / explanation:

- The IP range 203.0.113.0/24 is reserved for documentation and is an example, not a target.
- Reduce exposure by closing unnecessary services, restricting management interfaces, patching, using VPNs or allowlists, and monitoring cloud firewall rules.

Result: Public exposure information was reviewed and defensive actions were proposed.

Precautions: Do not use findings to access devices or attempt exploitation.

12. Hydra — Password-Attack Demonstration

Aim: Explain Hydra’s purpose and syntax and perform a limited test with approved test accounts.

Requirements: Hydra and an instructor-provided local training service/account.

Procedure:

- Confirm written authorization, target, service, test account, and attempt limit.
- Use the instructor’s exact lab configuration and a tiny test list.
- Observe logs and lockout/rate-limit behavior.
- Stop after the permitted attempts and document mitigation.

Commands / filters:

General syntax reference only: hydra -h

Record the lab command supplied by the instructor, target service, number of attempts, and observed result.

Observations / explanation:

- Hydra automates login attempts against supported protocols; unrestricted attempts can cause account lockouts or service load.
- Use long unique passwords, MFA, rate limiting, monitoring, and secure authentication protocols.

Result: The controlled password-testing concept and mitigation were documented.

Precautions: No credential lists or commands should be used against real services; use only instructor-approved test accounts and limits.

13. Dnsenum — DNS Information Gathering

Aim: Collect NS, MX, A, and AAAA information for an authorized domain.

Requirements: Dnsenum installed and an approved domain.

Procedure:

- Run a basic DNS enumeration on the approved domain.
- Record A, AAAA, NS, and MX results.

- Check the findings against the approved asset inventory.
- Document information exposure and remediation.

Commands / filters:

```
dnsenum example.com
dnsenum --help
```

Observations / explanation:

- Output varies by version and DNS configuration. Do not enable brute-force or zone-transfer testing unless the instructor specifically authorizes it.
- Keep DNS records current, remove stale records, restrict zone transfers to authorized secondaries, and monitor unexpected changes.

Result: DNS records were collected and interpreted.

Precautions: Stay within the approved domain and avoid aggressive enumeration options unless authorized.

14. Gobuster — Directory/File Discovery

Aim: Identify hidden files or directories on an authorized training website.

Requirements: Gobuster and an approved training web application.

Procedure:

- Select the instructor-approved URL and wordlist.
- Run a low-rate directory enumeration.
- Save output and review HTTP status codes.
- Validate findings and recommend defensive actions.

Commands / filters:

```
gobuster dir -u http://127.0.0.1/ -w /path/to/lab-wordlist.txt -o gobuster.txt
gobuster dir --help
```

Observations / explanation:

- The example targets localhost; replace the wordlist path with the supplied lab wordlist.
- This task belongs to web reconnaissance/content discovery in VAPT. Protect sensitive paths with server-side authorization, remove backups, and disable directory listing.

Result: Gobuster discovery was performed in the local lab and output was saved.

Precautions: Use only localhost or another explicitly authorized training target.

15. Burp Suite — DVWA Login Testing

Aim: Configure Burp proxy and inspect a controlled authentication request on local DVWA with dummy credentials.

Requirements: Local DVWA instance, Burp Suite, and dummy test account.

Procedure:

- Start DVWA locally and confirm the lab URL.
- Configure the browser proxy to Burp’s listener and open the local login page.
- Submit dummy credentials and inspect the request/response in HTTP history.
- Record parameter names, response behavior, and security observations without publishing secrets.

Commands / filters:

```
Example local URL: http://127.0.0.1/DVWA/
```

Observations / explanation:

- Verify HTTPS and secure cookie settings where applicable; protect session cookies; use CSRF protections where relevant, safe error messages, rate limiting, and MFA.
- Do not replay requests outside the local DVWA environment.

Result: The local authentication request was inspected and defensive observations were recorded.

Precautions: Use dummy credentials only; never capture or reuse real session tokens.

16. Linux File and Directory Management

Aim: Practice pwd, ls, cd, mkdir, cp, mv, rm, and find in a practice directory.

Requirements: Linux terminal.

Procedure:

- Create and enter a dedicated practice directory.
- Create sample files and folders.
- Copy and rename files, list contents, and search for a file.
- Remove only the practice files and record output.

Commands / filters:

pwd

```
mkdir -p ~/cyber-practice/demo
cd ~/cyber-practice/demo
```

printf "sample\n" > file1.txt

```
ls -la
cp file1.txt file2.txt
mv file2.txt renamed.txt
find . -type f
rm renamed.txt file1.txt
cd .. && rmdir demo
```

Observations / explanation:

- pwd prints the current directory; ls lists contents; cd changes directory; mkdir creates directories; cp copies; mv moves/renames; rm deletes; find searches for matching files.

Result: The required file and directory commands were practiced.

Precautions: Check the current path before using rm; avoid sudo and recursive deletion in practice exercises.

17. Linux File Permissions and User Management

Aim: Practice chmod, chown, whoami, id, groups, and ls -l and apply least privilege.

Requirements: Linux terminal and a practice file; chown may require administrator privileges.

Procedure:

- Create a practice file and inspect its permissions.
- Use chmod to grant owner read/write and group/other read access.
- Inspect the current user and group membership.
- If authorized, demonstrate chown on a lab-owned file.
- Explain why broad write permissions are unsafe.

Commands / filters:

```
touch permission-demo.txt
ls -l permission-demo.txt
chmod 640 permission-demo.txt
whoami
id
groups
chown USER:GROUP permission-demo.txt # only if authorized; replace placeholders
```

Observations / explanation:

- In mode 640, owner has read/write, group has read, and others have no permissions.
- Least privilege means each user/process receives only the permissions required for its task. Use narrowly scoped ownership and permissions.

Result: File permissions and identity/group commands were examined.

Precautions: Do not change ownership or permissions of system files; use only the practice file.

18. Command Injection on DVWA

Aim: Demonstrate command injection with a harmless test command in local DVWA and explain prevention.

Requirements: Local DVWA with the instructor-authorized command-injection module.

Procedure:

- Run DVWA locally and set the security level as directed by the instructor.
- Submit a normal test input and observe the baseline response.
- In the lab only, use the instructor-approved harmless command to demonstrate whether input is interpreted by a shell.
- Record evidence and restore the lab to its initial state.

Commands / filters:

Example harmless command concept: id

Use the instructor-provided input format for the local DVWA module; do not test external hosts.

Observations / explanation:

- Command injection occurs when untrusted input is concatenated into an operating-system command.
- Prefer safe library APIs over shell calls; avoid shell interpretation; use allowlists and strict input validation; run services with least privilege; log and monitor suspicious input.

Result: The vulnerability concept was demonstrated safely in local DVWA and mitigations were recorded.

Precautions: Never execute destructive commands or test real applications without explicit authorization.

19. Linux Process and System Monitoring

Aim: Use ps, top, free, df, du, uptime, and a safe test process to inspect system resources.

Requirements: Linux terminal.

Procedure:

- Inspect processes and system uptime.
- Review memory, filesystem space, and directory usage.
- Start a harmless sleep process and locate it.
- Stop only the process started for this exercise.

Commands / filters:

```
ps aux | head
top
free -h
df -h
du -sh ~
uptime
sleep 300 &
ps -ef | grep "[s]leep 300"
kill %1 # only if the sleep job is still the current shell job
```

Observations / explanation:

- ps lists processes; top shows live process/resource activity; free summarizes memory; df shows filesystem capacity; du estimates directory usage; uptime shows running time and load averages.
- Resource usage varies with workload and machine configuration.

Result: Process and system-resource commands were executed and interpreted.

Precautions: Terminate only the safe test process created for this practical.

20. Windows Command-Line Networking and Diagnostics

Aim: Practice ipconfig, ping, tracert, nslookup, netstat, and arp for network troubleshooting.

Requirements: Windows Command Prompt or PowerShell.

Procedure:

- Inspect local IP configuration.

- Test reachability to an approved host or gateway.
- Trace a route and query DNS.
- Inspect connections and the ARP cache.
- Record outputs and explain any failures.

Commands / filters:

```
ipconfig /all
ping 127.0.0.1
tracert example.com
nslookup example.com
netstat -ano
arp -a
```

Observations / explanation:

- ipconfig displays interface configuration; ping tests reachability; tracert shows route hops; nslookup queries DNS; netstat shows connections/listeners; arp displays IPv4-to-MAC cache entries.
- A failed ping alone does not prove a host is down because ICMP may be filtered.

Result: Windows networking commands were run and their outputs interpreted.

Precautions: Use approved hosts and avoid terminating processes based on netstat output unless instructed.

21. SQL Injection on DVWA and Hash Analysis

Aim: Test SQL injection with lab data in local DVWA and discuss secure queries; analyze sample hashes where appropriate.

Requirements: Local DVWA, dummy database data, and instructor-provided sample hashes.

Procedure:

- Open the SQL injection module in local DVWA at the instructor-assigned security level.
- Use only the lab's provided test inputs and observe how unsafe query construction changes the response.
- Compare vulnerable string-concatenated queries with parameterized queries conceptually.
- Analyze only supplied sample hashes using the approved hash-analysis workflow.
- Document impact and remediation.

Commands / filters:

```
Example safe SQL pattern (illustrative): SELECT * FROM users WHERE id = ?;
```

Use the lab-provided sample hashes only; see Practical 3 for the bounded John the Ripper workflow.

Observations / explanation:

- SQL injection can expose or modify database information or bypass intended checks.
- Use parameterized queries/prepared statements, least-privilege database accounts, safe error handling, and input validation as a secondary control.
- Store passwords with unique salts and a modern password-hashing scheme such as Argon2id, scrypt, or bcrypt.

Result: SQL injection behavior and sample hash risks were examined in the local training environment.

Precautions: Do not test production databases or use real user data.

22. OpenVAS / Greenbone — Target and Task Creation

Aim: Create a target and scan task for an authorized lab machine, select an available scan configuration, and record status.

Requirements: Greenbone/OpenVAS installation and an authorized lab target.

Procedure:

- Sign in to the Greenbone Security Assistant interface.
- Create a target using the lab machine IP and a suitable port list.
- Create a scan task, select the target and an available scan configuration recommended by the instructor.
- Start the task and monitor its status.
- Record target, configuration, start/end time, and status.

Commands / filters:

Record in the practical report: Target name/IP, port list, scan configuration, schedule, task status, and any scan errors.

Observations / explanation:

- Use a low-impact scan configuration for fragile systems and coordinate the scan window.
- A task can be queued, running, completed, or interrupted depending on resources and configuration.

Result: A target and scan task were configured and the status was recorded.

Precautions: Scan only the authorized lab machine; obtain approval before scanning production or shared networks.

23. OpenVAS Results Analysis and Report Generation

Aim: Review vulnerability findings, severity, affected hosts/ports, CVE references, remediation, and export a report.

Requirements: Completed Greenbone scan from Practical 22.

Procedure:

- Open the completed scan report.
- Sort findings by severity and review the affected host, port, detection evidence, and CVE references if present.
- Check whether findings are applicable and note any uncertainty or false-positive possibility.
- Prioritize fixes by severity, exposure, exploitability, and business impact.
- Export the report in an instructor-approved format.

Commands / filters:

Report fields: Scan name/date, target, finding title, severity/CVSS if provided, host/port, evidence, CVE, remediation, validation status, and retest result.

Observations / explanation:

- Verify high-impact findings before remediation decisions; scanners may produce false positives or miss issues.
- Patch affected software, disable unnecessary services, restrict network exposure, apply configuration fixes, and rescan to confirm remediation.

Result: Scan findings were reviewed, prioritized, and exported as a report.

Precautions: Treat scan reports as sensitive because they can reveal weaknesses in the lab or organization.

Suggested Practical Report Format

- Practical number and title
- Aim / objective
- Requirements and lab setup
- Procedure / commands used
- Screenshots or evidence captured in the lab
- Observations / sample output (label illustrative output clearly)
- Result / conclusion
- Security precautions and defensive recommendations

Submission reminder: Replace all example values with your actual lab observations and attach your own screenshots. Do not claim a scan, capture, or test was successful unless you performed it and observed the result.